รายงานโครงงาน

โปรแกรมจัดการคะแนนนักเรียน

นายภัทรพงศ์	โซวเจริญ	รหัสนักศึกษา 690331
นายอธิพัทธ์	สุทธิพล	รหัสนักศึกษา 690570
นายธนากร	ภวธนเกียรติ	รหัสนักศึกษา 691317
นายวรกร	ปริญวรวิช	รหัสนักศึกษา 691318

รายงานฉบับนี้เป็นส่วนหนึ่งของการเรียนวิชา CS1373

การพัฒนาโปรแกรมจัดการคะแนนนักเรียน

สาขาวิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยหัวเฉียวเฉลิมพระเกียรติ

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

รายงานเล่มนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของการเรียนวิชา CS1373 การพัฒนาโปรแกรม สาขาวิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยหัวเฉียวเฉลิมพระเกียรติ โดยมีวัตถุประสงค์เพื่อนำเสนอผลการดำเนินงานโครงงาน "ระบบจัดการคะแนนนักเรียนด้วยภาษา C" ซึ่งจัดทำขึ้นเพื่อแก้ปัญหาความล่าช้าและความผิดพลาดจากการคำนวณคะแนนและตัดเกรดด้วยมือ

เนื้อหาภายในรายงานฉบับนี้ประกอบด้วย การออกแบบโครงสร้างข้อมูลแบบ Array สำหรับจัดเก็บข้อมูลนักเรียน การแบ่งฟังก์ชันการทำงาน เพื่อความมีระเบียบของระบบ ตลอดจนการพัฒนาฟังก์ชันคำนวณเกรด คะแนนเฉลี่ย การเรียงลำดับคะแนน ค้นหาข้อมูลด้วยอัลกอริทึม Binary Search และการสรุปสถิติภาพรวมในห้องเรียน ซึ่งช่วยอำนวยความสะดวกให้ผู้สอนสามารถจัดการข้อมูลได้อย่างแม่นยำและรวดเร็ว

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่า รายงานและโปรแกรมฉบับนี้จะเป็นประโยชน์แก่ครู อาจารย์ นักศึกษา และผู้ที่สนใจศึกษาการเขียนโปรแกรมด้วยภาษา C ไม่มากก็น้อย หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้ด้วย

คณะผู้จัดทำ

สารบัญ

เรื่อง	**หน้า**
คำนำ	(ก)
สารบัญ	(ข-ค)
บทที่ 1 ที่มาและความสำคัญของโครงงาน	1-2
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์ของโครงงาน	1
1.3 ขอบเขตของโครงงาน	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 ทฤษฎี และเครื่องมือที่เกี่ยวข้อง	3-8
2.1 หลักการเขียนโปรแกรมแบบแยกโมดูล	3
2.2 โครงสร้างข้อมูลแบบระเบียนและแถบอาร์เรย์	4
2.3 ตัวแปรชี้ตำแหน่งและการส่งค่าผ่านแอดเดรส	5
2.4 โครงสร้างควบคุมและการวนซ้ำ	6
2.5 อัลกอริทึมการค้นหาและการเรียงลำดับ	7
2.6 ตัวแปลภาษาและไลบรารีมาตรฐาน	8
บทที่ 3 การวิเคราะห์และออกแบบระบบ	9-18
3.1 การวิเคราะห์ความต้องการของระบบ	9
3.2 การออกแบบโครงสร้างข้อมูล	10
3.3 ตารางสรุปหน้าที่ของฟังก์ชัน	10
3.4 ผังงานการทำงานของระบบและฟังก์ชันย่อย	11-18
บทที่ 4 การพัฒนาระบบ	19-23
4.1 โครงสร้างข้อมูลและตัวแปรหลัก	19
4.2 ฟังก์ชันเมนูหลักและการควบคุมทิศทาง	20
4.3 ฟังก์ชันจัดการข้อมูลนักเรียน	20-21
4.4 ฟังก์ชันค้นหาและเรียงลำดับ	22
4.5 ฟังก์ชันตัดเกรดและสรุปสถิติ	23

สารบัญ (ต่อ)

เรื่อง	หน้า
บทที่ 5 ผลการดำเนินงาน	24-26
5.1 สรุปผลการดำเนินงาน	24
5.2 อภิปรายผล	25
5.3 ข้อจำกัดของระบบ	25
5.4 ข้อเสนอแนะในการพัฒนาต่อยอด	26
ภาคผนวก	27
คู่มือการใช้งานระบบ	28-30
ภาพเบื้องหลังการทำงาน	31
หน้าที่รับผิดชอบในการทำงานของสมาชิกแต่ละคน	32

บทที่ 1
ที่มาและความสำคัญของโครงงาน

1.1 ที่มาและความสำคัญ

ในการจัดการเรียนการสอน ผู้สอนหรืออาจารย์มักประสบปัญหาเรื่องระยะเวลาในการตรวจนับและคิดคำนวณคะแนนรวม ตลอดจนการตัดเกรดของนักเรียนแต่ละคน หากใช้วิธีคำนวณด้วยมือแบบดั้งเดิม อาจใช้เวลานานและมีโอกาสเกิดข้อผิดพลาดในการคำนวณได้ง่าย

คณะผู้จัดทำจึงได้เล็งเห็นปัญหานี้ และได้พัฒนา **โปรแกรมระบบจัดการคะแนนนักเรียนด้วยภาษา C** ขึ้นมา เพื่อช่วยให้อาจารย์และเจ้าหน้าที่สามารถทำงานได้สะดวกและรวดเร็วยิ่งขึ้น โดยระบบจะเข้ามาช่วยจัดการข้อมูลนักเรียน คำนวณเกรด คะแนนเฉลี่ย ค้นหาคะแนนสูงสุด-ต่ำสุด ตลอดจนสรุปสถิติภาพรวมให้อัตโนมัติ ซึ่งช่วยลดข้อผิดพลาดจากการคำนวณด้วยมือ และเพิ่มประสิทธิภาพในการทำงานให้ดียิ่งขึ้น

1.2 วัตถุประสงค์ของโครงงาน

1. **เพื่อฝึกฝนทักษะการเขียนโปรแกรมภาษา C** โดยประยุกต์ใช้โครงสร้างข้อมูลแบบ Array, การแบ่งฟังก์ชัน (Function), คำสั่งวนซ้ำ (Loop) และการใช้เงื่อนไขต่าง ๆ
2. **เพื่อพัฒนาระบบจัดเก็บ ค้นหา คำนวณเกรด คะแนนเฉลี่ย และประมวลผลสถิติ** ของนักเรียนให้สามารถใช้งานได้ง่ายและแม่นยำ

1.3 ขอบเขตของโครงงาน

1. **ขอบเขตด้านการทำงาน**: ระบบออกแบบมาเพื่ออำนวยความสะดวกแก่อาจารย์ผู้สอนและเจ้าหน้าที่ในการเพิ่มข้อมูล ค้นหา ตรวจสอบรายชื่อ คำนวณเกรด และสรุปสถิติคะแนนนักเรียนผ่านหน้าจอ Console
2. **ข้อจำกัดของระบบ**: โปรแกรมเวอร์ชันนี้ยังทำงานในลักษณะ In-Memory และ**ยังไม่รองรับระบบฐานข้อมูลถาวร** (Database) ระบบล็อกอินเข้าสู่ระบบ (Authentication) การเชื่อมต่อเว็บไซต์ และการบันทึกข้อมูลถาวร

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. **ช่วยอำนวยความสะดวก** แก่อาจารย์ผู้สอนและเจ้าหน้าที่ในการจัดการ ค้นหา และประมวลผลคะแนนได้อย่างรวดเร็ว
2. **ช่วยลดข้อผิดพลาด** ที่อาจเกิดขึ้นจากการคำนวณเกรดและค่าสถิติต่าง ๆ ด้วยมือ
3. **ช่วยพัฒนาทักษะ** ของคณะผู้จัดทำในการเขียนโปรแกรมภาษา C ผ่านการใช้งาน Array และ Function ในการแก้ปัญหาจริง

บทที่ 2
ทฤษฎี และเครื่องมือที่เกี่ยวข้อง

ในการพัฒนา **โปรแกรมระบบจัดการคะแนนนักเรียนด้วยภาษา C** คณะผู้จัดทำได้ศึกษาและประยุกต์ใช้หลักการเขียนโปรแกรม โครงสร้างข้อมูล อัลกอริทึม และเครื่องมือพัฒนาซอฟต์แวร์ โดยมีรายละเอียดทางทฤษฎีพร้อมตัวอย่างการอ้างอิงโค้ดในโครงงาน ดังนี้

2.1 หลักการเขียนโปรแกรมแบบแยกโมดูล

ทฤษฎี:

การเขียนโปรแกรมแบบแยกโมดูล คือการแตกซอฟต์แวร์ขนาดใหญ่ออกเป็นฟังก์ชันย่อยๆ (Functions) หลายๆ ตัวที่มีหน้าที่เฉพาะเจาะจง เพื่อให้โค้ดมีความเป็นระเบียบ ง่ายต่อการอ่าน ทดสอบ บำรุงรักษา และลดความซ้ำซ้อนของโค้ด

ตัวอย่างการประยุกต์ใช้ในโครงงาน:

โปรแกรมได้แบ่งการทำงานออกเป็นฟังก์ชันย่อย และมีการประกาศ Prototype ไว้ที่ส่วนหัวของโปรแกรม เช่น:

```
// ตัวอย่างการประกาศ Prototype ของฟังก์ชันในโปรแกรม
int showMenu();
void addStudent();
void removeStudent();
void showStudent();
int findStudent(int* id);
void sortByScore(struct Student S[], int n, int t);
float calculateAverage(struct Student s[], int n);
char calculateGrade(float score);
```

คำอธิบาย: แต่ละฟังก์ชันจะรับผิดชอบงานเฉพาะ เช่น addStudent() ทำหน้าที่รับข้อมูลนักเรียน, calculateGrade() ทำหน้าที่คำนวณเกรด และ showMenu() ทำหน้าที่แสดงหน้าจอตัวเลือก ทำให้โค้ดในฟังก์ชัน main() มีความกระชับและอ่านเข้าใจได้ง่าย

2.2 โครงสร้างข้อมูลแบบระเบียนและแถบอาร์เรย์

ทฤษฎี:

1. **โครงสร้างข้อมูลแบบระเบียน (struct):** เป็นการสร้างชนิดข้อมูลใหม่ที่สามารถรวบรวมตัวแปรต่างชนิดกัน (Heterogeneous Data Types) เข้าไว้ด้วยกันเป็นกลุ่มเดียว
2. **แถบอาร์เรย์ (Array):** เป็นโครงสร้างข้อมูลเชิงเส้นที่จัดเก็บกลุ่มของข้อมูลชนิดเดียวกันไว้ในหน่วยความจำที่เรียงต่อกันเป็นบล็อกคงที่ โดยใช้อินเด็กซ์ (Index) ในการเข้าถึงข้อมูล

ตัวอย่างการประยุกต์ใช้ในโครงงาน:

ในโครงงานนี้ได้ประยุกต์ใช้ **อาร์เรย์ของสตรักเจอร์ (Array of Structs)** เพื่อจัดเก็บข้อมูลนักเรียนทุกคน

```
#define MAX_STUDENTS 100
// การสร้างชนิดข้อมูล
struct Student struct Student {
        int id; // รหัสนักเรียน (Integer)
        char name; // ชื่อ-นามสกุล (Character Array)
        float score; // คะแนนสอบ (Float)
};
// การประกาศอาร์เรย์และตัวแปรกำหนดขนาด
struct Student students[MAX_STUDENTS];
unsigned int studentCount = 0;
```

คำอธิบาย: struct Student รวมข้อมูล ID, Name และ Score ของนักเรียน 1 คนไว้ในโหนดเดียว จากนั้นประกาศเป็นอาร์เรย์ students[MAX_STUDENTS] เพื่อจองพื้นที่ในหน่วยความจำสำหรับเก็บข้อมูลนักเรียนได้สูงสุด 100 คน โดยมีตัวแปร studentCount เป็นตัวนับจำนวนรายการข้อมูลที่มีจริง

2.3 ตัวแปรชี้ตำแหน่งและการส่งค่าผ่านแอดเดรส

ทฤษฎี:

ตัวแปรพอยน์เตอร์ (Pointer) คือตัวแปรที่ใช้จัดเก็บที่อยู่บนหน่วยความจำ (Memory Address) ของตัวแปรอื่น การส่งค่าเข้าฟังก์ชันด้วยพอยน์เตอร์เป็นการส่งแบบ Pass-by-Reference ซึ่งช่วยให้ฟังก์ชันสามารถเข้าถึงและแก้ไขค่าในตำแหน่งหน่วยความจำจริงได้โดยไม่ต้องคัดลอกข้อมูลใหม่

ตัวอย่างการประยุกต์ใช้ในโครงงาน:

ฟังก์ชันค้นหานักเรียน findStudent() รับพารามิเตอร์เป็นพอยน์เตอร์ int* id

```
// การประกาศฟังก์ชันรับค่าแบบ Pointer
int findStudent(int* id) {
        ...
        if (students[mid].id == *id) {
                // เข้าถึงค่าในแอดเดรสด้วย Dereference operator (*)
                return mid;
        }
        ...
}
// ตัวอย่างการเรียกใช้งานในฟังก์ชันอื่น
int check = findStudent(&id); // ส่งตำแหน่งแอดเดรสด้วย Address-of operator (&)
```

คำอธิบาย: ฟังก์ชันรับพารามิเตอร์เป็นแอดเดรส int* id และส่งแอดเดรส &id เข้าไป ทำให้การเปรียบเทียบค่ารหัสนักเรียนทำได้โดยตรงผ่านแอดเดรส เพิ่มประสิทธิภาพในการประมวลผล

2.4 โครงสร้างควบคุมและการวนซ้ำ

ทฤษฎี:

โครงสร้างควบคุมคือคำสั่งที่ใช้ตัดสินใจเลือกทิศทางการทำงานของโปรแกรม (if-else, switch-case) และคำสั่งวนซ้ำ (do-while, for) เพื่อประมวลผลข้อมูลซ้ำๆ ตามเงื่อนไข

ตัวอย่างการประยุกต์ใช้ในโครงงาน:

1. **โครงสร้างเมนูและการเลือกทำ (do-while & switch-case):**

```
do {
        system("cls");
        choice = showMenu();
        switch (choice) {
                case 1: addStudent(); break;
                case 2: removeStudent(); break;
                ...
                case 9: break;
        }
} while (choice != 9);
```

คำอธิบาย: ใช้ do-while เพื่อแสดงเมนูซ้ำไปเรื่อยๆ จนกว่าผู้ใช้จะเลือกเมนู 9 เพื่อออกจากโปรแกรม และใช้ switch-case เพื่อแยกทิศทางการทำงานไปยังฟังก์ชันต่างๆ

2. **การตัดเกรดด้วยเงื่อนไขหลายชั้น (if-else):**

```
char calculateGrade(float score) {
        if (score >= 80) return 'A';
        else if (score >= 70) return 'B';
        else if (score >= 60) return 'C';
        else if (score >= 50) return 'D';
        else return 'F';
}
```

คำอธิบาย: ตรวจสอบช่วงคะแนนทีละระดับเพื่อแปลงค่าคะแนนสอบชนิด float เป็นเกรดอักษรชนิด char

3. **การขยับตำแหน่งอาร์เรย์ด้วยวนซ้ำ (for loop):**

```
for (int i = idx; i < studentCount - 1; i++) {
        students[i] = students[i + 1];
}
```

คำอธิบาย: ในฟังก์ชัน removeStudent() ใช้ลูป for เลื่อนข้อมูลอาร์เรย์ตัวถัดไปมาทับตำแหน่งที่ต้องการลบเพื่อปิดช่องว่างในอาร์เรย์

2.5 อัลกอริทึมการค้นหาและการเรียงลำดับ

ทฤษฎี:

1. **Binary Search Algorithm**: อัลกอริทึมค้นหาข้อมูลแบบแบ่งครึ่ง ทำงานบนข้อมูลที่จัดเรียงลำดับแล้ว มีประสิทธิภาพยุทธวิธีแบ่งแยกและเอาชนะ (Divide and Conquer) โดยมีซับซ้อนของเวลาเป็น O(log n)
2. **Bubble Sort Algorithm**: อัลกอริทึมเรียงลำดับข้อมูลโดยเปรียบเทียบข้อมูลคู่ที่อยู่ติดกันแล้วทำการสลับตำแหน่ง (Swap) ไปเรื่อยๆ จนกว่าข้อมูลจะเรียงลำดับครบทั้งหมด

ตัวอย่างการประยุกต์ใช้ในโครงงาน:

1. **การค้นหาด้วย Binary Search ใน findStudent():**

```
sortStudent(); // เรียงลำดับ ID จากน้อยไปมากก่อนเสมอ
while (low <= high && !flag) {
        mid = (low + high) / 2;
        if (students[mid].id == *id) return mid;
        else if (students[mid].id < *id) low = mid + 1;
        else high = mid - 1;
}
```

คำอธิบาย: ก่อนทำการค้นหา ต้องเรียกใช้ sortStudent() เพื่อเรียง ID นักเรียนก่อน จากนั้นคำนวณตำแหน่งกลาง mid เพื่อลดช่วงการค้นหาลงทีละครึ่งหนึ่ง ทำให้ค้นหาได้รวดเร็ว

2. **การเรียงลำดับด้วย Bubble Sort ใน sortByScore():**

```
for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
                if (t == 0 && S[j].score < S[j + 1].score) {
                        // สลับเมื่อเรียงจากมากไปน้อย
                        temp = S[j];
                        S[j] = S[j + 1];
                        S[j + 1] = temp;
                }
        }
}
```

คำอธิบาย: ใช้ตัวแปรชั่วคราว struct Student temp ในการสลับตำแหน่งสตรักเจอร์นักเรียนทั้งโหนด ตามเงื่อนไขทิศทางการเรียงลำดับคะแนนที่ผู้ใช้เลือก

2.6 ตัวแปลภาษาและไลบรารีมาตรฐาน

ทฤษฎี:

ภาษา C ทำงานร่วมกับพรีโพรเซสเซอร์ (Preprocessor Directives) เพื่อดึงไลบรารีมาตรฐานมาใช้งาน เช่น การจัดการ Input/Output, การจัดการสตริง และคำสั่งควบคุมหน้าจอ Console

ตัวอย่างการประยุกต์ใช้ในโครงงาน:

```
#include <stdio.h> // สำหรับ printf(), scanf(), fgets()
#include <string.h> // สำหรับ strcpy(), strcspn()
#include <conio.h> // สำหรับ getch()
#include <stdlib.h> // สำหรับ system("cls")
```

คำอธิบาย: เรียกใช้ fgets() และ strcspn() จาก <string.h> ในฟังก์ชัน addStudent() เพื่อรับค่าชื่อเว้นวรรคและตัดอักขระขึ้นบรรทัดใหม่ รวมถึงใช้ system("cls") จาก <stdlib.h> และ getch() จาก <conio.h> ในการล้างและหยุดรอหน้าจอ Console

บทที่ 3

การวิเคราะห์และออกแบบระบบ

การวิเคราะห์และออกแบบระบบเป็นขั้นตอนสำคัญในการกำหนดโครงสร้างการทำงาน เพื่อให้โปรแกรมจัดการคะแนนนักเรียนสามารถประมวลผลได้อย่างถูกต้อง มีระเบียบ และรองรับการใช้งานผ่านหน้าจอ Console ได้อย่างมีประสิทธิภาพ โดยมีรายละเอียดการออกแบบดังนี้

3.1 การวิเคราะห์ความต้องการของระบบ

ระบบได้รับการออกแบบเพื่อรองรับความต้องการใช้งานของผู้สอนและเจ้าหน้าที่ ดังนี้

1. **ความต้องการเชิงฟังก์ชัน:**
   - **การจัดการข้อมูลนักเรียน:** ระบบต้องสามารถเพิ่มรายชื่อนักเรียนใหม่ (รหัส ชื่อ-นามสกุล คะแนน) โดยตรวจสอบไม่ให้รหัสนักเรียนซ้ำกัน และสามารถลบข้อมูลนักเรียนออกจากอาร์เรย์ได้
   - **การแสดงผลและการค้นหา:** ระบบต้องแสดงตารางรายชื่อนักเรียนพร้อมเกรดที่ได้รับ และสามารถค้นหาข้อมูลนักเรียนรายบุคคลด้วยรหัสนักเรียนได้อย่างรวดเร็ว
   - **การเรียงลำดับและการประมวลผลสถิติ:** ระบบต้องสามารถเรียงลำดับนักเรียนตามคะแนนสอบ (จากมากไปน้อยหรือน้อยไปมาก) ตลอดจนคำนวณคะแนนรวม ค่าเฉลี่ย และตัดเกรดอัตโนมัติ
2. **ความต้องการเชิงเทคนิค:**
   - ระบบทำงานแบบสวิตช์เมนูเลือกรายการ (Interactive Menu Loop) ผ่านหน้าจอ Console บนสภาพแวดล้อม Dev-C++
   - กำหนดพื้นที่จัดเก็บข้อมูลนักเรียนสูงสุด 100 รายการด้วยโครงสร้างอาร์เรย์คงที่ (MAX_STUDENTS 100)

3.2 การออกแบบโครงสร้างข้อมูล

- **struct Student**:
  - int id : รหัสนักเรียน
  - char name[40] : ชื่อ-นามสกุล
  - float score : คะแนนสอบ
- **Global Variables**:
  - struct Student students[100] : อาร์เรย์จัดเก็บข้อมูล
  - unsigned int studentCount : ตัวแปรนับจำนวนรายการปัจจุบัน

3.3 การออกแบบโครงสร้างเมนูและฟังก์ชัน

โปรแกรมออกแบบตามแนวทาง Modular Programming โดยแบ่งหน้าที่การทำงานออกเป็นฟังก์ชันย่อย ดังนี้

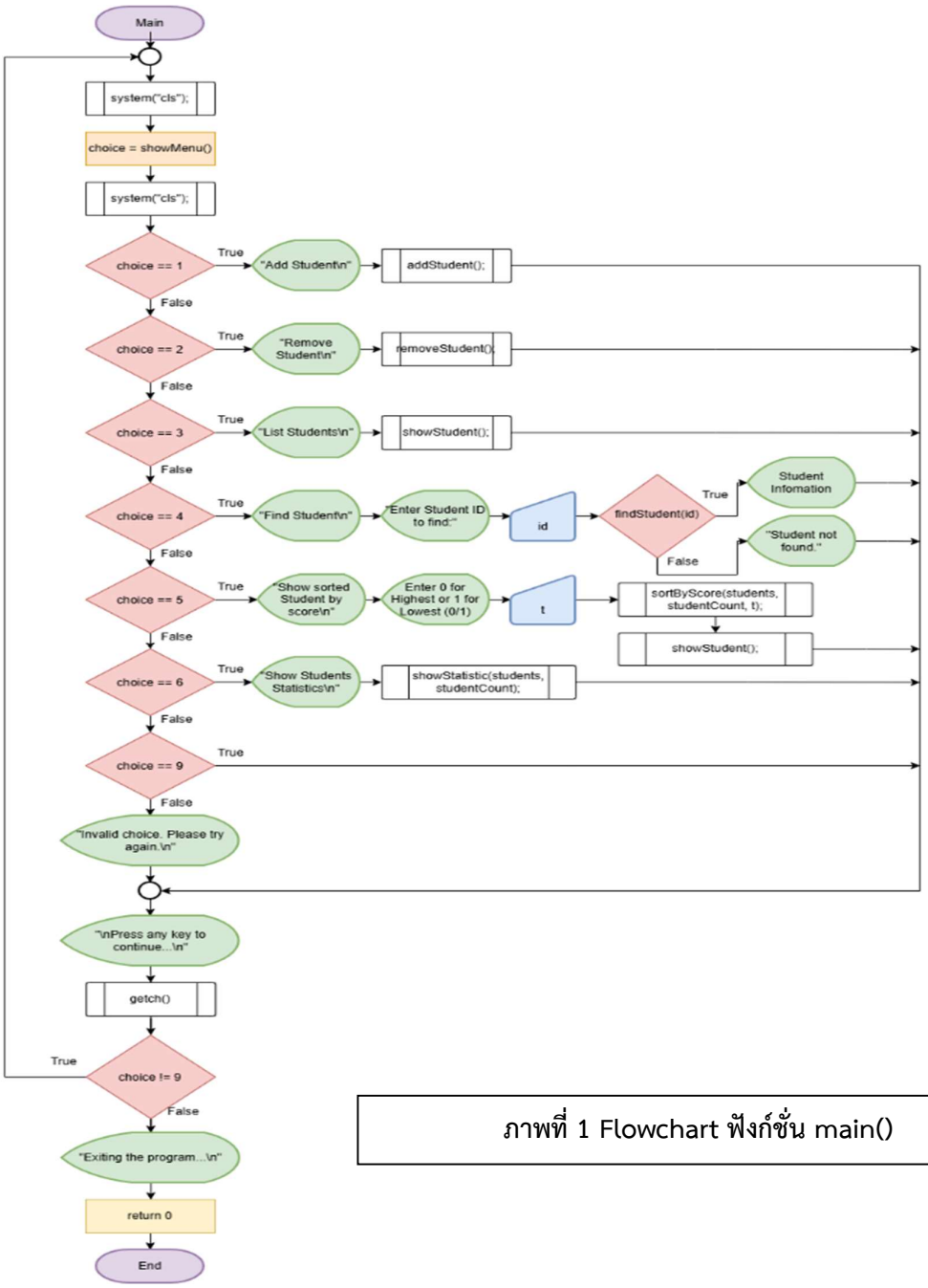
ชื่อฟังก์ชัน	หน้าที่และความรับผิดชอบ	ผู้รับผิดชอบ
showMenu()	แสดงเมนูหลัก (1-6 และ 9) รับค่าทางเลือกจากผู้ใช้	นายภัทรพงศ์ โซวเจริญ
addStudent()	รับรหัส ชื่อ คะแนน และเพิ่มลงในอาร์เรย์ (ตรวจสอบ ID ซ้ำ)	นายอธิพัทธ สุทธิพล
removeStudent()	ลบนักเรียนตาม ID โดยการขยับตำแหน่งสมาชิกในอาร์เรย์	นายอธิพัทธ สุทธิพล
showStudent()	แสดงตารางรายชื่อนักเรียน รหัส คะแนน และเกรดทั้งหมด	นายวรกร ปริญวรวิช
findStudent()	ค้นหาตำแหน่งนักเรียนด้วย Binary Search (จัดเรียง ID ก่อนค้นหา)	นายภัทรพงศ์ โซวเจริญ
sortStudent()	เรียงลำดับข้อมูลนักเรียนตามรหัสนักเรียนด้วย Bubble Sort	นายภัทรพงศ์ โซวเจริญ
sortByScore()	เรียงลำดับนักเรียนตามคะแนน (มากไปน้อย / น้อยไปมาก) ด้วย Bubble Sort	นายวรกร ปริญวรวิช
showStatistic()	แสดงรายชื่อ สรุปคะแนนรวม และค่าเฉลี่ยคะแนนของชั้นเรียน	นายวรกร ปริญวรวิช
calculateAverage()	คำนวณค่าเฉลี่ยคะแนนสอบของนักเรียนทั้งหมด	นายธนากร ภวธนเกียรติ
calculateGrade()	แปลงคะแนนสอบเป็นเกรดอักษร (A, B, C, D, F)	นายธนากร ภวธนเกียรติ

ตารางที่ 1 การออกแบบโครงสร้างเมนูและฟังก์ชัน

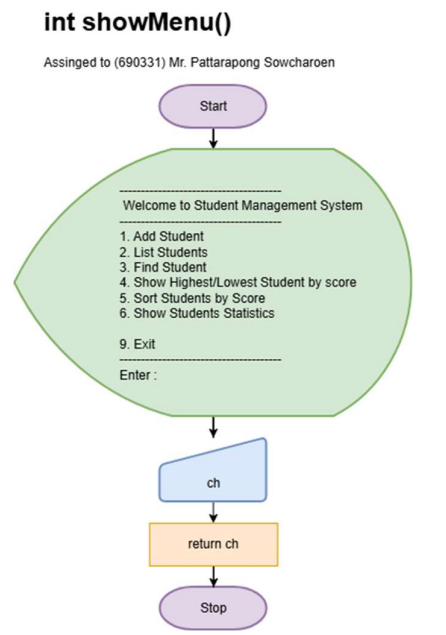
3.4 ผังงานการทำงานของระบบและฟังก์ชันย่อย

3.4.1 ผังงานโปรแกรมหลัก

- ขั้นตอนการทำงาน:
  1. เริ่มต้นโปรแกรม (Start)
  2. เรียกใช้ showMenu() แสดงตัวเลือก (1-6, 9)
  3. ตรวจสอบค่า choice ด้วย switch-case เพื่อเรียกใช้ฟังก์ชันที่เกี่ยวข้อง
  4. ทำงานซ้ำด้วยวนซ้ำ do-while จนกระทั่งผู้ใช้เลือกเมนู 9 เพื่อจบการทำงาน



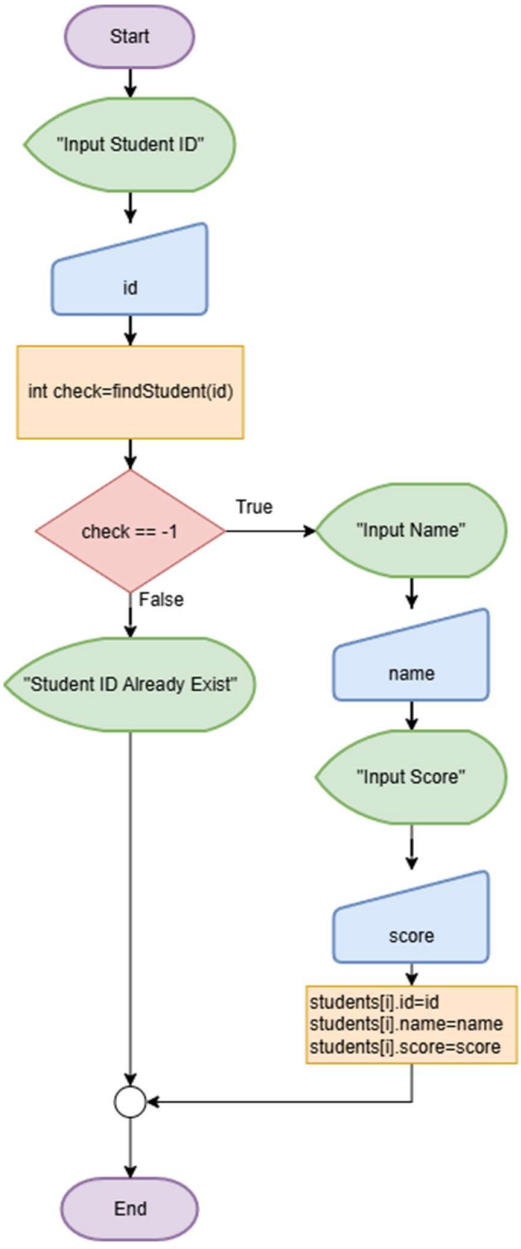
ภาพที่ 1 Flowchart ฟังก์ชั่น main()



ภาพที่ 2 Flowchart ฟังก์ชั่น showMenu()

3.4.2 ผังงานฟังก์ชันเพิ่มข้อมูลนักเรียน (addStudent)

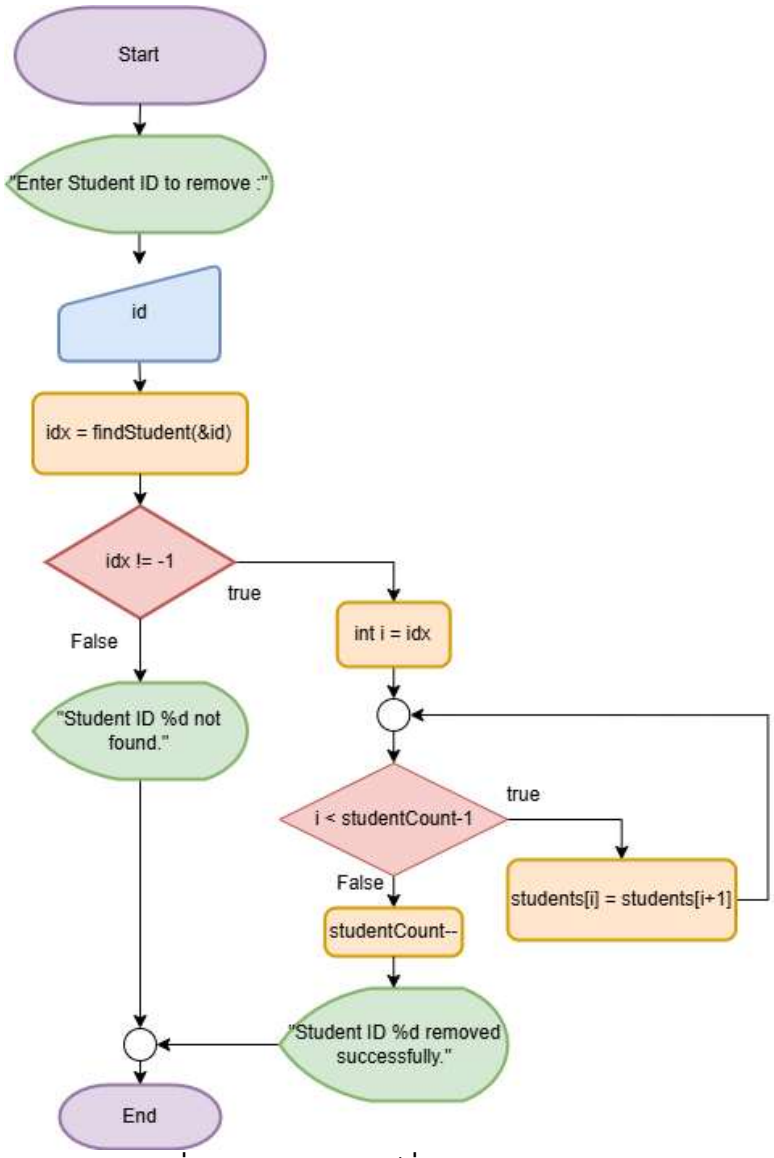
- ขั้นตอนการทำงาน:
  1. รับค่า id จากผู้ใช้
  2. เรียก findStudent(&id) เพื่อตรวจสอบว่ามี ID นี้ในระบบหรือไม่
  3. **ถ้าพบ ID ซ้ำ (check != -1)**: แสดงข้อความ *"Student ID Already Exist"* และกลับสู่เมนู
  4. **ถ้าไม่พบ ID ซ้ำ (check == -1)**: รับค่า name และ score บันทึกลง students[studentCount] แล้วเพิ่มค่า studentCount++



ภาพที่ 3 Flowchart ฟังก์ชั่น addStudent()

3.4.3 ผังงานฟังก์ชันลบข้อมูลนักเรียน (removeStudent)

- ขั้นตอนการทำงาน:
  1. รับค่า id จากผู้ใช้
  2. เรียก findStudent(&id) เพื่อตรวจสอบว่ามี ID นี้ในระบบหรือไม่
  3. ถ้าไม่พบข้อมูล (idx == -1): แสดงข้อความ *"Student ID not found"*
  4. ถ้าพบข้อมูล (idx != -1): ใช้ลูป for วนขยับข้อมูลตัวถัดไปมาทับตำแหน่งเดิม (students[i] = students[i+1]) แล้วลดค่า studentCount—

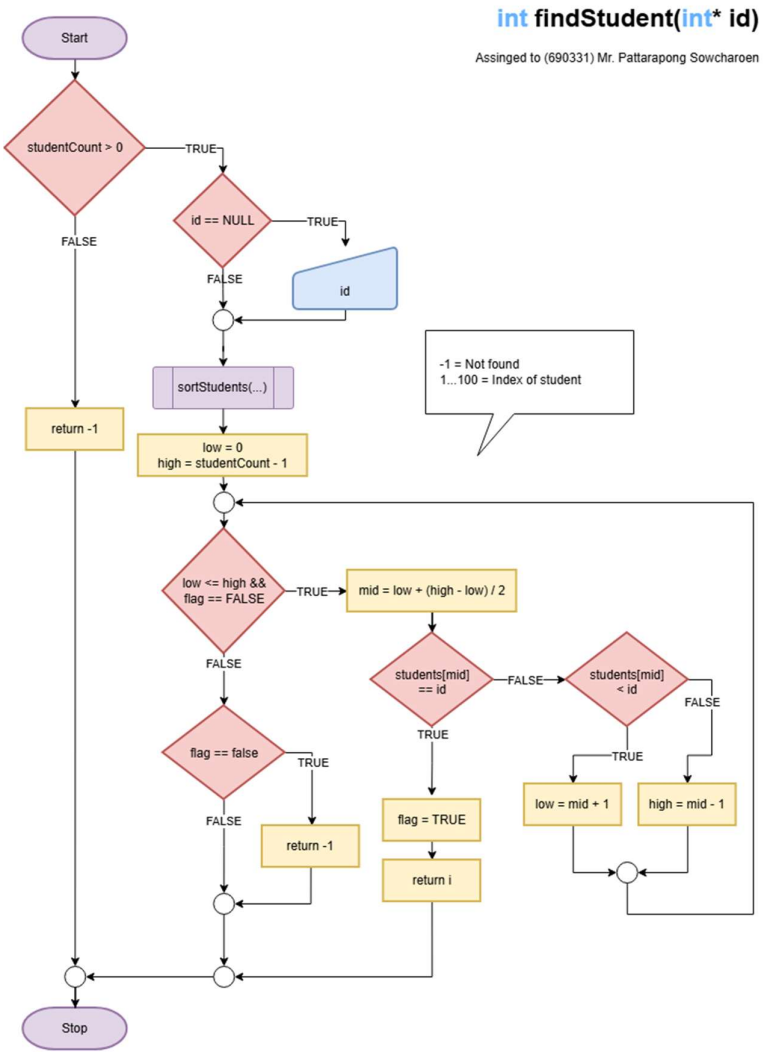


ภาพที่ 4 Flowchart ฟังก์ชั่น removeStudent()

3.4.4 ผังงานฟังก์ชันค้นหาข้อมูลด้วย Binary Search (findStudent)

- ขั้นตอนการทำงาน:

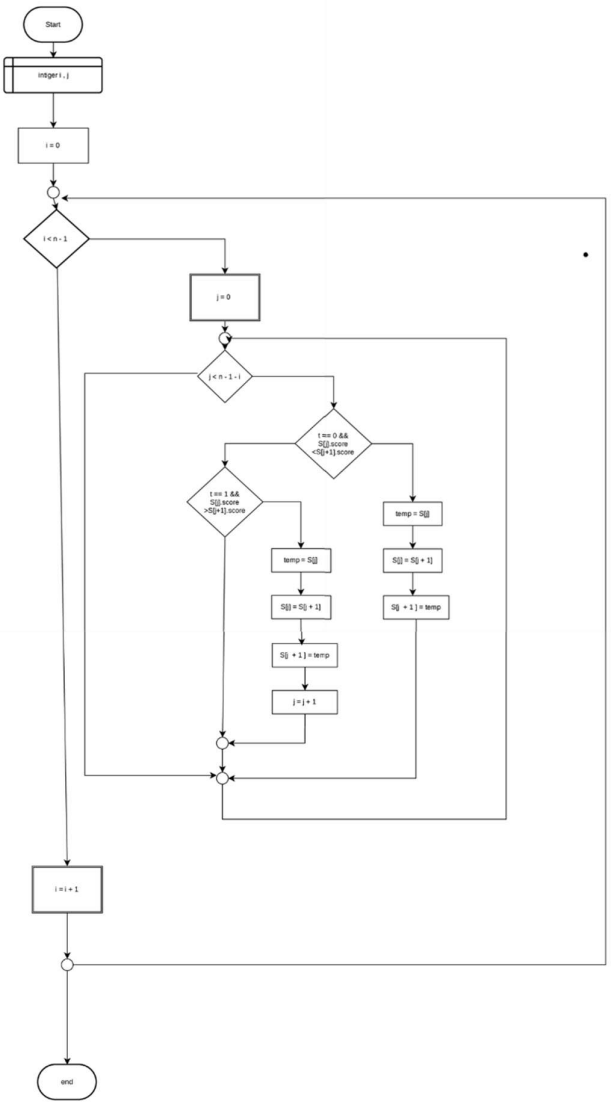
1. ตรวจสอบว่ามีข้อมูลในระบบหรือไม่ (studentCount > 0)
2. เรียก sortStudent() เพื่อเรียงลำดับ ID จากน้อยไปมากก่อนเสมอ
3. กำหนดตำแหน่ง low = 0 และ high = studentCount - 1
4. วนลูปคำนวณ mid = (low + high) / 2
5. เปรียบเทียบ students[mid].id == targetID:
   - ถ้าเท่ากัน คืนค่าตำแหน่ง mid
   - ถ้าน้อยกว่า ปรับ low = mid + 1
   - ถ้ามากกว่า ปรับ high = mid – 1
6. หากหาไม่พบจนจบช่วงลูป คืนค่า -1



ภาพที่ 5 Flowchart ฟังก์ชั่น findStudent()

3.4.5 ผังงานฟังก์ชันเรียงลำดับคะแนน (sortByScore)

- ขั้นตอนการทำงาน:
  1. รับอาร์เรย์ S[], จำนวนนักเรียน n, และประเภทการเรียง t (0 = มากไปน้อย, 1 = น้อยไปมาก)
  2. วนลูปซ้อนด้วย Bubble Sort (for i และ for j)
  3. เปรียบเทียบเงื่อนไขคะแนนคู่ติดกัน:
     - หาก t == 0 และคะแนนตัวหน้า < คะแนนตัวหลัง ให้สลับตำแหน่ง
     - หาก t == 1 และคะแนนตัวหน้า > คะแนนตัวหลัง ให้สลับตำแหน่ง
  4. อัปเดตการเรียงลำดับในอาร์เรย์

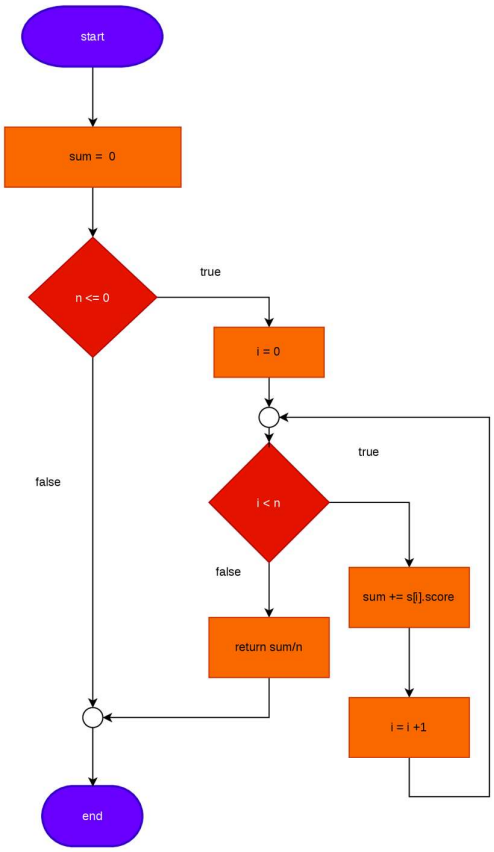


ภาพที่ 6 Flowchart ฟังก์ชั่น sortByScore()

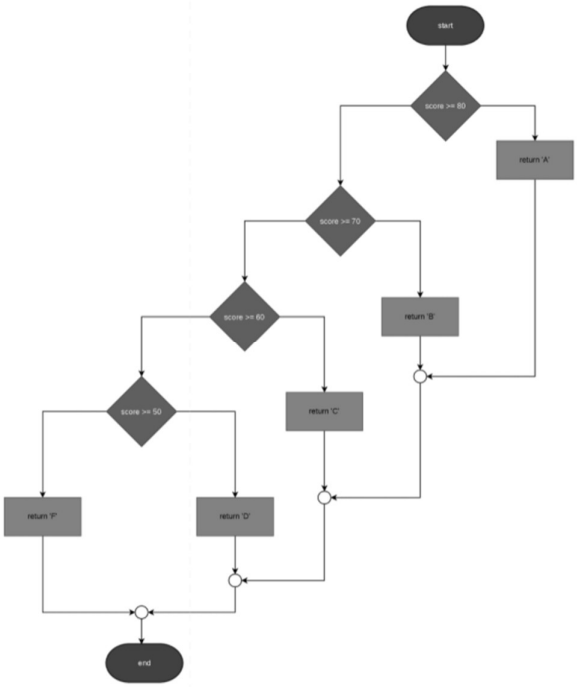
3.4.6 ผังงานฟังก์ชันประมวลผลสถิติและตัดเกรด (showStatistic & calculateGrade)

- ขั้นตอนการทำงาน:

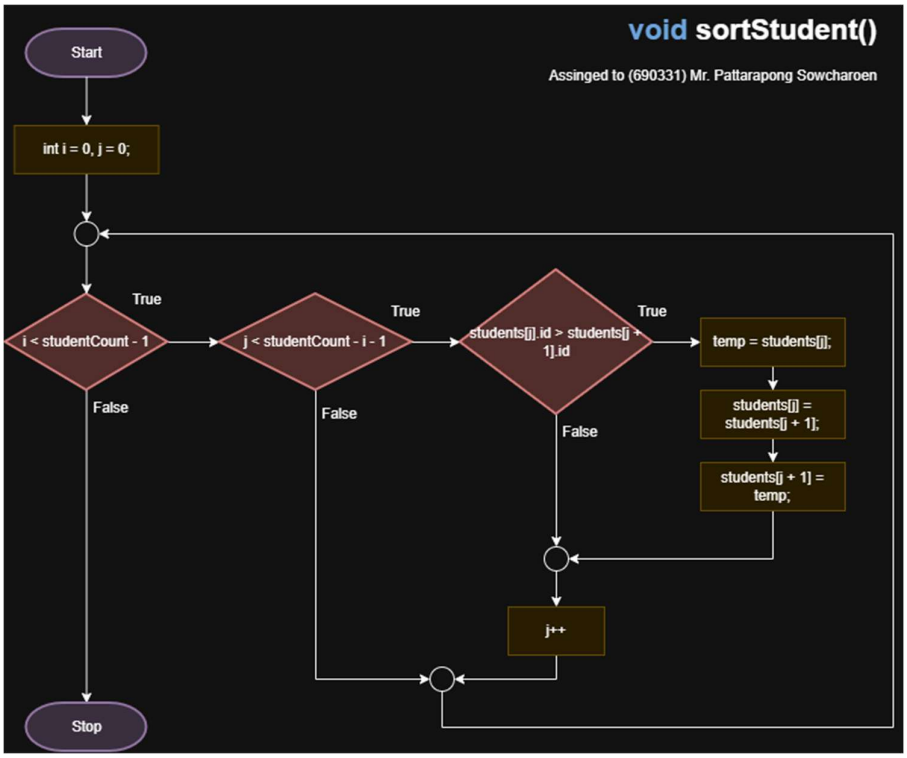
1. **calculateGrade(score)**: ใช้เงื่อนไข if-else ตรวจสอบช่วงคะแนน (≥80 ➔ 'A', ≥70 ➔ 'B', ≥60 ➔ 'C', ≥50 ➔ 'D', <50 ➔ 'F')
2. **showStatistic()**: วนลูปสะสมคะแนนรวม total += score นำมาหารด้วย n เพื่อหา avg แล้วแสดงผลตารางสรุปคะแนนพร้อมเกรดของนักเรียนทุกคน



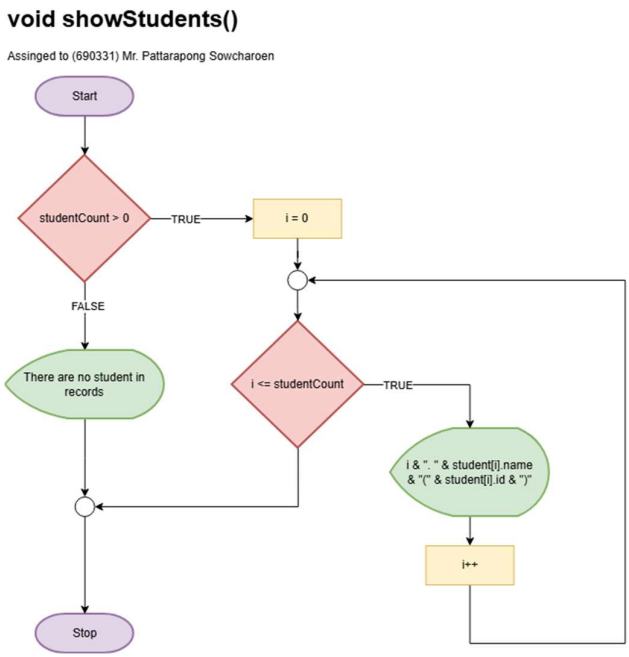
ภาพที่ 7 Flowchart ฟังก์ชั่น calculateAvg()



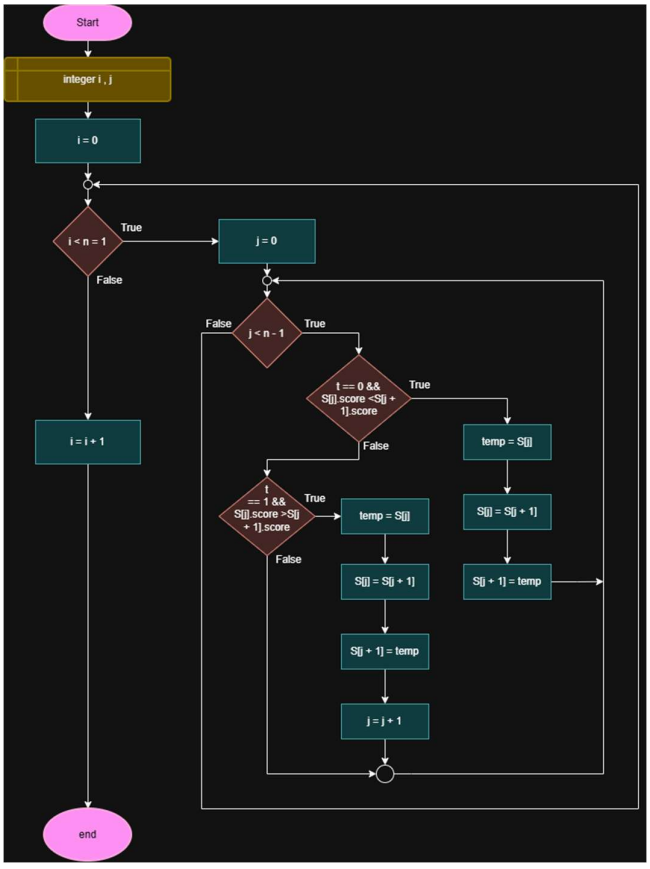
ภาพที่ 8 Flowchart ฟังก์ชั่น calculateGrade()



ภาพที่ 9 Flowchart ฟังก์ชั่น sortStudent() by Id



ภาพที่ 10 Flowchart ฟังก์ชั่น showStudent()



ภาพที่ 11 Flowchart ฟังก์ชั่น sortByScore()

บทที่ 4

การพัฒนาระบบ

ในบทนี้จะเป็นการนำเสนอรายละเอียดซอร์สโค้ดส่วนสำคัญของระบบจัดการคะแนนนักเรียนด้วยภาษา C พร้อมอธิบายจุดสำคัญและหลักการทำงานของแต่ละฟังก์ชัน เพื่อแสดงให้เห็นถึงการทำงานของซอฟต์แวร์ที่ได้พัฒนาขึ้น

4.1 โครงสร้างข้อมูลและตัวแปรหลัก

โปรแกรมเริ่มด้วยการกำหนดโครงสร้างข้อมูล Student และอาร์เรย์ระดับ Global เพื่อจัดเก็บข้อมูลในหน่วยความจำ

```
#define MAX_STUDENTS 100
struct Student {
        int id; // รหัสนักเรียน
        char name[40]; // ชื่อ-นามสกุล
        float score; // คะแนนสอบ
};
struct Student students[MAX_STUDENTS];
unsigned int studentCount = 0;
```

- **จุดสำคัญ:**
    - กำหนดขนาดอาร์เรย์สูงสุดไว้ที่ 100 รายการด้วย #define MAX_STUDENTS 100
    - studentCount ทำหน้าที่นับจำนวนข้อมูลนักเรียนที่มีอยู่ในปัจจุบัน และใช้เป็นดรรชนีในการเพิ่มข้อมูลตำแหน่งถัดไป

4.2 ฟังก์ชันเมนูหลักและการควบคุมทิศทาง (main & showMenu)

ควบคุมการทำงานด้วยเมนูโต้ตอบผ่านวนซ้ำ do-while และคำสั่ง switch-case

```
int showMenu() {
        int ch;
        printf("1. Add Student\n2. Remove Student\n3. List Students\n");
        printf("4. Find Student\n5. Show sorted Student by score\n");
        printf("6. Show Students Statistics\n\n9. Exit\n");
        printf("Enter : ");
        scanf("%d", &ch);
        return ch;
}
```

- **จุดสำคัญ:**
    - showMenu() แสดงรายการตัวเลือก และคืนค่าตัวเลขที่ผู้ใช้กรอกกลับไปยัง main()
    - ใช้ฟังก์ชัน system("cls") ล้างหน้าจอ Console ก่อนและหลังทำแต่ละรายการ เพื่อให้หน้าจอเป็นระเบียบ อ่านง่าย

4.3 ฟังก์ชันจัดการข้อมูลนักเรียน (addStudent & removeStudent)

1. **การเพิ่มข้อมูลนักเรียน (addStudent)**

```
void addStudent() {
        int id;
        char name[40];
        float score;
        printf("Input Student ID: ");
        scanf("%d", &id);
        int check = findStudent(&id); // ตรวจสอบ ID ซ้ำ
        if (check == -1) {
                getchar();
                printf("Input Name: ");
                fgets(name, sizeof(name), stdin);
                name[strcspn(name, "\n")] = '\0'; // ลบ newline character
                printf("Input Score: ");
                scanf("%f", &score);
```

```
            students[studentCount].id = id;
            strcpy(students[studentCount].name, name);
            students[studentCount].score = score;
            studentCount++;
        } else {
            printf("\nStudent ID Already Exist");
        }
}
```

- **จุดสำคัญ:**
    - เรียกใช้ findStudent(&id) ก่อนทำการบันทึก เพื่อป้องกันการกรอกรหัสนักเรียนซ้ำกัน
    - ใช้ fgets() ในการรับค่าชื่อ-นามสกุล เพื่อรองรับการพิมพ์เว้นวรรค และใช้ strcspn() ตัดอักขระขึ้นบรรทัดใหม่ (\n) ออก

2. **การลบข้อมูลนักเรียน (removeStudent)**

```
void removeStudent() {
        int id, idx;
        printf("Enter Student ID to remove: ");
        scanf("%d", &id);
        idx = findStudent(&id);
        if (idx != -1) {
            for (int i = idx; i < studentCount - 1; i++) {
                students[i] = students[i + 1]; // ขยับสมาชิกตำแหน่งถัดไปมาทับ
            }
            studentCount--;
            printf("\nStudent ID %d removed successfully.\n", id);
        } else {
            printf("\nStudent ID %d not found.\n", id);
        }
}
```

- **จุดสำคัญ:**
    - ใช้การวนลูปขยับข้อมูลอาร์เรย์ทางขวาขึ้นมาแทนที่ตำแหน่งที่ถูกลบ (students[i] = students[i + 1]) แล้วลดค่า studentCount ลง 1

4.4 ฟังก์ชันค้นหาและเรียงลำดับ (findStudent & sortByScore)

1. **การค้นหาด้วย Binary Search (findStudent)**

```
int findStudent(int* id) {
	int low = 0, mid, high = studentCount - 1;
	bool flag = false;
	if (!(studentCount > 0)) return -1;
	sortStudent(); // เรียงลำดับ ID จากน้อยไปมากก่อนทำ Binary Search
	while (low <= high && !flag) {
		mid = (low + high) / 2;
		if (students[mid].id == *id) {
			return mid; // พบข้อมูล คืนค่า Index
		} else if (students[mid].id < *id) {
			low = mid + 1;
		} else { high = mid - 1; }
	}
	return -1; // ไม่พบข้อมูล
}
```

- **จุดสำคัญ:**
  - เรียก sortStudent() ก่อนค้นหาเสมอ เนื่องจากอัลกอริทึม **Binary Search** ต้องทำบนข้อมูลที่เรียงลำดับแล้วเท่านั้น
  - ช่วยให้ค้นหาข้อมูลได้อย่างรวดเร็ว โดยแบ่งครึ่งช่วงค้นหาในทุกรอบลูป

2. **การเรียงลำดับตามคะแนนสอบ (sortByScore)**

```
void sortByScore(struct Student S[], int n, int t) {
	struct Student temp;
	for (int i = 0; i < n - 1; i++) {
		for (int j = 0; j < n - 1 - i; j++) {
			if ((t == 0 && S[j].score < S[j + 1].score) ||
			    (t == 1 && S[j].score > S[j + 1].score)) {
			    temp = S[j];
			    S[j] = S[j + 1];
			    S[j + 1] = temp;
			}
		}
	}
}
```

- **จุดสำคัญ:**
  - ประยุกต์ใช้อัลกอริทึม **Bubble Sort** โดยรับพารามิเตอร์ t เพื่อเลือกทิศทางการเรียงคะแนนได้ทั้งแบบมากไปน้อย (t=0) และน้อยไปมาก (t=1)

4.5 ฟังก์ชันตัดเกรดและสรุปสถิติ (calculateGrade & showStatistic)

```
char calculateGrade(float score) {
        if (score >= 80) return 'A';
        else if (score >= 70) return 'B';
        else if (score >= 60) return 'C';
        else if (score >= 50) return 'D';
        else return 'F';
}
void showStatistic(struct Student S[], int n) {
        float total = 0, avg;
        for (int i = 0; i < n; i++) {
                total += S[i].score;
        }
        avg = total / n;
        printf("Total Score : %.1f\nAverage Score : %.2f\n", total, avg);
}
```

- **จุดสำคัญ:**
    - calculateGrade() เป็นฟังก์ชันบริสุทธิ์ (Pure Function) คืนค่าอักษรเกรดตามเกณฑ์คะแนนที่กำหนด
    - showStatistic() วนลูปคำนวณคะแนนรวมและค่าเฉลี่ยของนักเรียนทุกคนในห้องเรียนเพื่อนำเสนอภาพรวมสถิติ

บทที่ 5
ผลการดำเนินงาน

การพัฒนาโปรแกรมระบบจัดการคะแนนนักเรียนด้วยภาษา C ผ่านโปรแกรม Dev-C++ สำเร็จลุล่วงตามขอบเขตและวัตถุประสงค์ที่ตั้งไว้ คณะผู้จัดทำจึงขอนำเสนอสรุปผลการดำเนินงาน อภิปรายผล ข้อจำกัด และข้อเสนอแนะในการพัฒนาต่อยอด ดังนี้

5.1 สรุปผลการดำเนินงาน

ระบบจัดการคะแนนนักเรียนได้รับการทดสอบฟังก์ชันการทำงานผ่านหน้าจอ Console พบว่าสามารถประมวลผลข้อมูลได้ถูกต้องและราบรื่นตามเป้าหมาย โดยผลการดำเนินงานจำแนกตามฟังก์ชันหลักได้ดังนี้

1. **การจัดการข้อมูลนักเรียน (addStudent & removeStudent):**

ระบบสามารถเพิ่มข้อมูลนักเรียน (รหัส ชื่อ คะแนน) ลงในอาร์เรย์ได้ถูกต้อง โดยมีระบบป้องกันการกรอกรหัสนักเรียนซ้ำ และสามารถลบข้อมูลนักเรียนตาม ID โดยขยับตำแหน่งสมาชิกในอาร์เรย์ได้อย่างแม่นยำ

2. **การค้นหาและเรียงลำดับข้อมูล (findStudent & sortByScore):**

ระบบค้นหารหัสนักเรียนด้วยอัลกอริทึม **Binary Search** ร่วมกับการจัดเรียง ID ล่วงหน้าได้อย่างรวดเร็ว อีกทั้งสามารถเรียงลำดับคะแนนสอบของนักเรียนทั้งแบบคะแนนสูงสุดไปต่ำสุดและต่ำสุดไปสูงสุดด้วย **Bubble Sort** ได้ตามต้องการ

3. **การตัดเกรดและประมวลผลสถิติ (calculateGrade & showStatistic):**

ระบบสามารถตัดเกรดอักษร (A, B, C, D, F) อัตโนมัติตามเกณฑ์คะแนนที่กำหนด และคำนวณคะแนนรวมพร้อมค่าเฉลี่ยของห้องเรียนเพื่อนำเสนอในรูปแบบตารางสรุปสถิติได้อย่างถูกต้อง

5.2 อภิปรายผล

จากการทดสอบใช้งานโปรแกรม พบว่าระบบช่วยลดระยะเวลาและความซ้ำซ้อนในการทำงานของผู้สอนเมื่อเปรียบเทียบกับการคำนวณคะแนนด้วยมือแบบดั้งเดิม อีกทั้งยังช่วยลดโอกาสเกิดข้อผิดพลาดในการคำนวณคะแนนเฉลี่ยและการตัดเกรด นอกจากนี้ การพัฒนาโครงงานนี้ยังช่วยให้คณะผู้จัดทำได้ฝึกฝนการประยุกต์ใช้ทฤษฎีภาษา C เช่น Struct, Array, Function, Loop และ Pointers ในการแก้ปัญหาจริงได้อย่างมีประสิทธิภาพ

5.3 ข้อจำกัดของระบบ

แม้ว่าโปรแกรมจะสามารถทำงานได้ตามวัตถุประสงค์หลัก แต่ยังมีข้อจำกัดที่ควรปรับปรุง ดังนี้

1. **การจัดเก็บข้อมูลชั่วคราว (In-Memory Storage):**

โปรแกรมจัดเก็บข้อมูลนักเรียนไว้ในอาร์เรย์ในหน่วยความจำชั่วคราว (RAM) และจำกัดจำนวนนักเรียนสูงสุดไว้ที่ 100 รายการ (`MAX_STUDENTS 100`) โดยยังไม่มีการเชื่อมต่อฐานข้อมูลหรือไฟล์บันทึกข้อมูลถาวร ทำให้ข้อมูลสูญหายเมื่อปิดโปรแกรม

2. **ขอบเขตการใช้งานและส่วนติดต่อผู้ใช้:**

ระบบทำงานผ่านหน้าจอ Console แบบข้อความเท่านั้น ยังไม่มีส่วนติดต่อผู้ใช้แบบกราฟิก (GUI) หรือเว็บอินเทอร์เฟซ

3. **ความปลอดภัยของระบบ:**

ยังไม่มีระบบตรวจสอบตัวตนหรือสิทธิ์การเข้าใช้งาน (User Authentication/Login) เพื่อจำกัดการเข้าถึงข้อมูล

5.4 ข้อเสนอแนะในการพัฒนาต่อยอด

เพื่อให้ระบบมีความสมบูรณ์และรองรับการใช้งานในระดับที่สูงขึ้น ควรพิจารณาพัฒนาต่อยอดในประเด็นต่อไปนี้

1. **การเพิ่มระบบบันทึกข้อมูลถาวร (File I/O / Database):**

ควรพัฒนาให้โปรแกรมสามารถอ่านและเขียนไฟล์ (เช่น ไฟล์ .txt, .csv) หรือเชื่อมต่อระบบฐานข้อมูล (Database) เพื่อให้สามารถบันทึกและโหลดข้อมูลนักเรียนกลับมาใช้งานใหม่ได้ทุกครั้ง

2. **การใช้โครงสร้างข้อมูลแบบยืดหยุ่น (Dynamic Memory Allocation):**

ควรเปลี่ยนจากการใช้อาร์เรย์ขนาดคงที่ (MAX_STUDENTS 100) ไปใช้การจัดสรรหน่วยความจำแบบไดนามิก (เช่น Linked List หรือ Dynamic Array) เพื่อรองรับจำนวนนักเรียนได้อย่างไม่จำกัด

3. **การพัฒนาส่วนติดต่อผู้ใช้ (GUI / Web Application):**

ปรับเปลี่ยนหน้าจอ Console ให้เป็น Graphical User Interface (GUI) หรือพัฒนาเป็น Web-based Application เพื่อเพิ่มความสะดวกในการใช้งาน

4. **w:**

เพิ่มระบบลงชื่อเข้าใช้งาน (Login) พร้อมกำหนดสิทธิ์การเข้าถึง เช่น แยกสิทธิ์ระหว่างอาจารย์ผู้สอน (จัดการข้อมูลได้) และนักเรียน (ดูผลคะแนนได้อย่างเดียว)

ภาคผนวก

คู่มือการใช้งานระบบ

คู่มือฉบับนี้จัดทำขึ้นเพื่ออธิบายขั้นตอนการติดตั้ง สภาพแวดล้อมการทำงาน และวิธีการใช้งานโปรแกรมระบบจัดการคะแนนนักเรียนด้วยภาษา C ผ่านหน้าจอ Console

1. การเตรียมความพร้อมก่อนใช้งาน

1. พร้อมใช้งาน

โหลด Pre-Build จาก Github (สำหรับ Windows x64 เท่านั้น

https://github.com/pattr-hcu/student-management

2. คอมไพล์เอง

1. เครื่องมือที่จำเป็น:

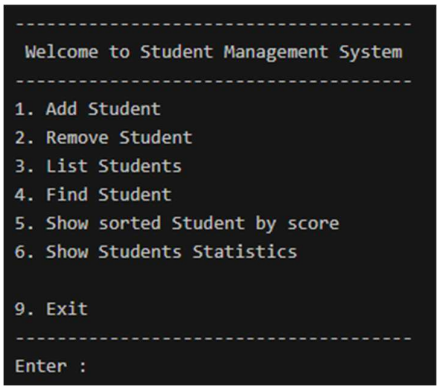
โปรแกรม Dev-C++ (หรือ C Compiler เช่น Code::Blocks, GCC Compiler)

2. ขั้นตอนการเปิดโปรแกรม:

- เปิดไฟล์ซอร์สโค้ด main.c ในโปรแกรม Dev-C++
- กดปุ่ม F11 (Compile & Run) เพื่อแปลภาษาและเริ่มสั่งประมวลผลโปรแกรม
- หน้าจอ Console จะแสดงเมนูหลักของโปรแกรมขึ้นมา

2. โครงสร้างเมนูหลักของโปรแกรม

เมื่อโปรแกรมเริ่มทำงาน จะปรากฏเมนูหลักจำนวน 7 ตัวเลือกดังนี้



ภาพที่ 12 หน้าต่างการทำงานฟังก์ชั่น showMenu()

3. คำแนะนำและขั้นตอนการใช้งานแต่ละฟังก์ชัน

1. การเพิ่มข้อมูลนักเรียนใหม่

- **วิธีการใช้งาน**

1. พิมพ์ตัวเลข 1 แล้วกด Enter
2. ในช่อง Input Student ID: ให้พิมพ์ **รหัสนักเรียน** (ตัวเลข) แล้วกด **Enter**
3. ระบบจะตรวจสอบรหัสนักเรียน:
   - **หากรหัสซ้ำ:** แสดงข้อความ Student ID Already Exist และกลับสู่เมนูหลัก
   - **หากรหัสไม่ซ้ำ:** ระบบจะให้พิมพ์ **Input Name:** (ชื่อ-นามสกุล) และ **Input Score:** (คะแนนสอบ 0-100) ตามลำดับ
4. เมื่อกรอกครบ ข้อมูลจะถูกบันทึกลงในระบบทันที

2. การลบข้อมูลนักเรียน

- **วิธีการใช้งาน**

1. พิมพ์ตัวเลข 2 แล้วกด **Enter**
2. ใพิมพ์รหัสนักเรียนที่ต้องการลบในช่อง Enter Student ID to remove: แล้วกด **Enter**
3. ผลลัพธ์:
   - **พบเรคอร์ด: ระบบทำการลบข้อมูลออกจากอาร์เรย์ และแสดงข้อความ Student ID [ID] removed successfully.**
   - **ไม่พบข้อมูล: แสดงข้อความ Student ID [ID] not found.**

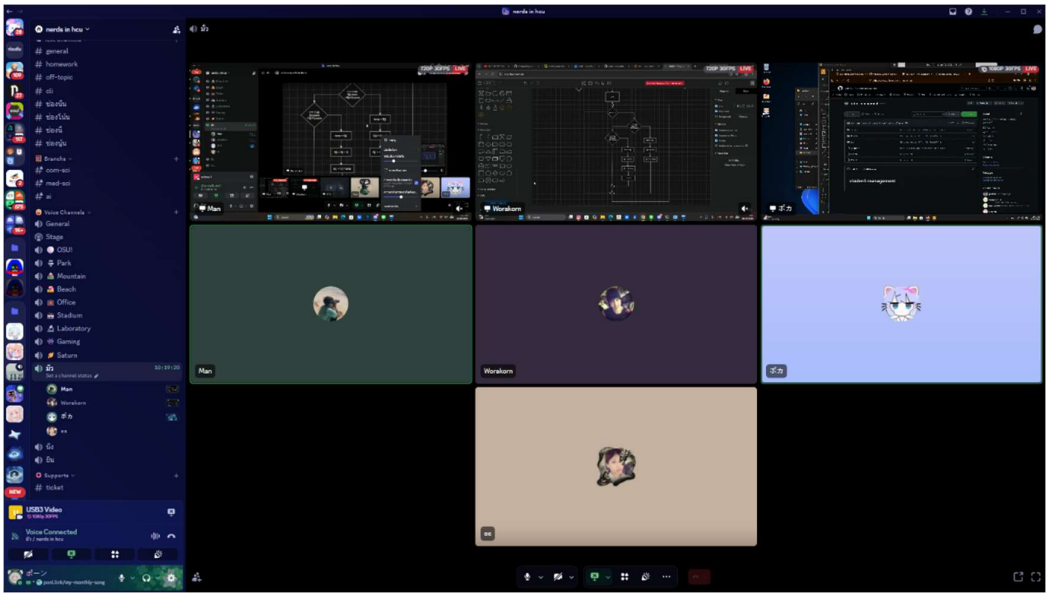
3. การแสดงตารางรายชื่อนักเรียนทั้งหมด

- **วิธีการใช้งาน**

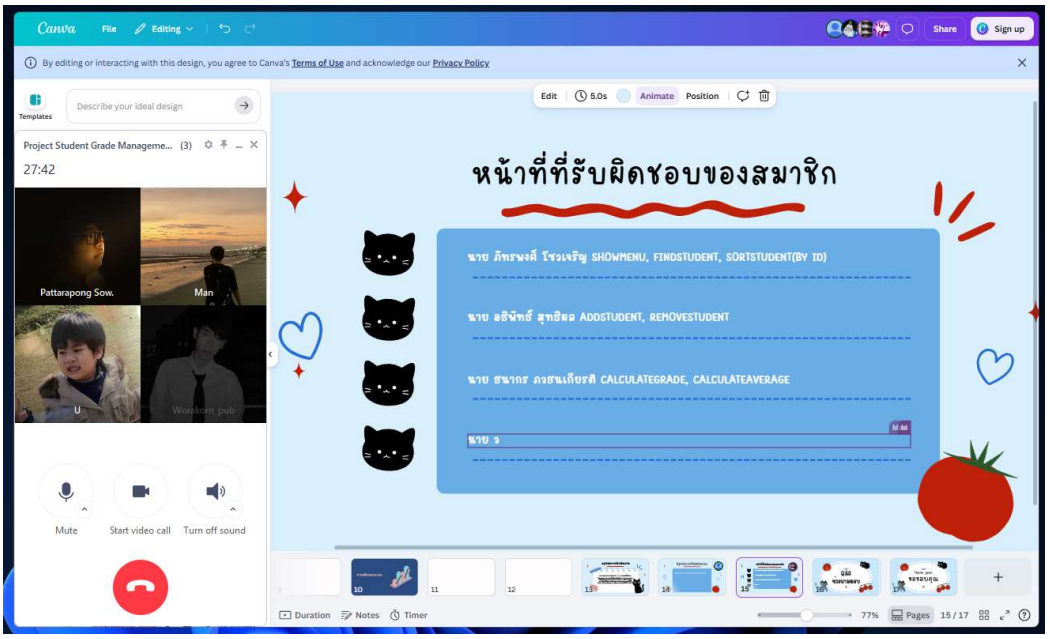
1. พิมพ์ตัวเลข 3 แล้วกด **Enter**
2. ระบบจะแสดงตารางจัดเรียงข้อมูลนักเรียนทุกคนในระบบ ประกอบด้วย: **ID**, **Name**, **Score** และ **Grade** (คำนวณเกรด A, B, C, D, F ให้อัตโนมัติ)
3. หากยังไม่มีข้อมูลนักเรียนในระบบ จะแสดงข้อความ There are no student in record.

4. **การค้นหาข้อมูลนักเรียนด้วยรหัส**
    - **วิธีการใช้งาน**
        1. พิมพ์ตัวเลข 4 แล้วกด Enter
        2. พิมพ์รหัสนักเรียนในช่อง Enter Student ID to find: แล้วกด **Enter**
        3. โปรแกรมจะทำการเรียงลำดับ ID และค้นหาด้วยอัลกอริทึม **Binary Search**:
            - **หากพบข้อมูล: แสดงรายละเอียด ID, Name, Score และ Grade ของนักเรียนรายนั้น**
            - **หากไม่พบข้อมูล: แสดงข้อความ Student ID [ID] not found.**
5. **การเรียงลำดับนักเรียนตามคะแนนสอบ**
    - **วิธีการใช้งาน**
        1. พิมพ์ตัวเลข 5 แล้วกด Enter
        2. เลือกรูปแบบการเรียงลำดับในช่อง Enter 0 for Highest or 1 for Lowest (0/1):
            - **พิมพ์ 0 : เรียงลำดับจาก คะแนนมากไปน้อย (Highest to Lowest)**
            - **พิมพ์ 1 : เรียงลำดับจาก คะแนนน้อยไปมาก (Lowest to Highest)**
        3. ระบบจะแสดงตารางรายชื่อนักเรียนที่จัดเรียงคะแนนตามประเภทที่เลือก
6. **การแสดงสถิติคะแนนภาพรวมของห้องเรียน**
    - **วิธีการใช้งาน**
        1. พิมพ์ตัวเลข 6 แล้วกด Enter
        2. ระบบจะแสดงตารางรายชื่อนักเรียนพร้อมคะแนนและเกรดทุกคน และคำนวณสรุปสถิติท้ายตาราง ได้แก่:
            - **Total Score: คะแนนรวมของนักเรียนทุกคนในห้องเรียน**
            - **Average Score: คะแนนเฉลี่ยของห้องเรียน**
7. **การออกจากโปรแกรม**
    - **วิธีการใช้งาน**
        1. พิมพ์ตัวเลข 9 แล้วกด Enter
        2. โปรแกรมจะแสดงข้อความ Exiting the program... และปิดการทำงานลงอย่างสมบูรณ์

ภาพเบื้องหลังการทำงาน



ภาพที่ 13 การประชุม และทำงานพร้อมกันผ่านโปรแกรม Discord



ภาพที่ 14 การประชุม และทำงานพร้อมกันผ่านโปรแกรม Line

หน้าที่รับผิดชอบในการทำงานของสมาชิกแต่ละคน

สมาชิก	ฟังก์ชั่นที่รับผิดชอบ
1. นายภัทรพงศ์ โชวเจริญ 690331	showMenu, findStudent, sortStudent (by id)
2. นายอธิพัทธ์ สุทธิพล 690570	AddStudent, RemoveStudent
3. นายธนากร ภวธนเกียรติ 691317	CalculateGrade, CalculateAverage
4. นายวรกร ปริญวรวิช 691318	SortStudent (by score), ShowStatistic, showStudent

ตารางที่ 2 หน้าที่รับผิดชอบฟังก์ชั่นของสมาชิกแต่ละคน